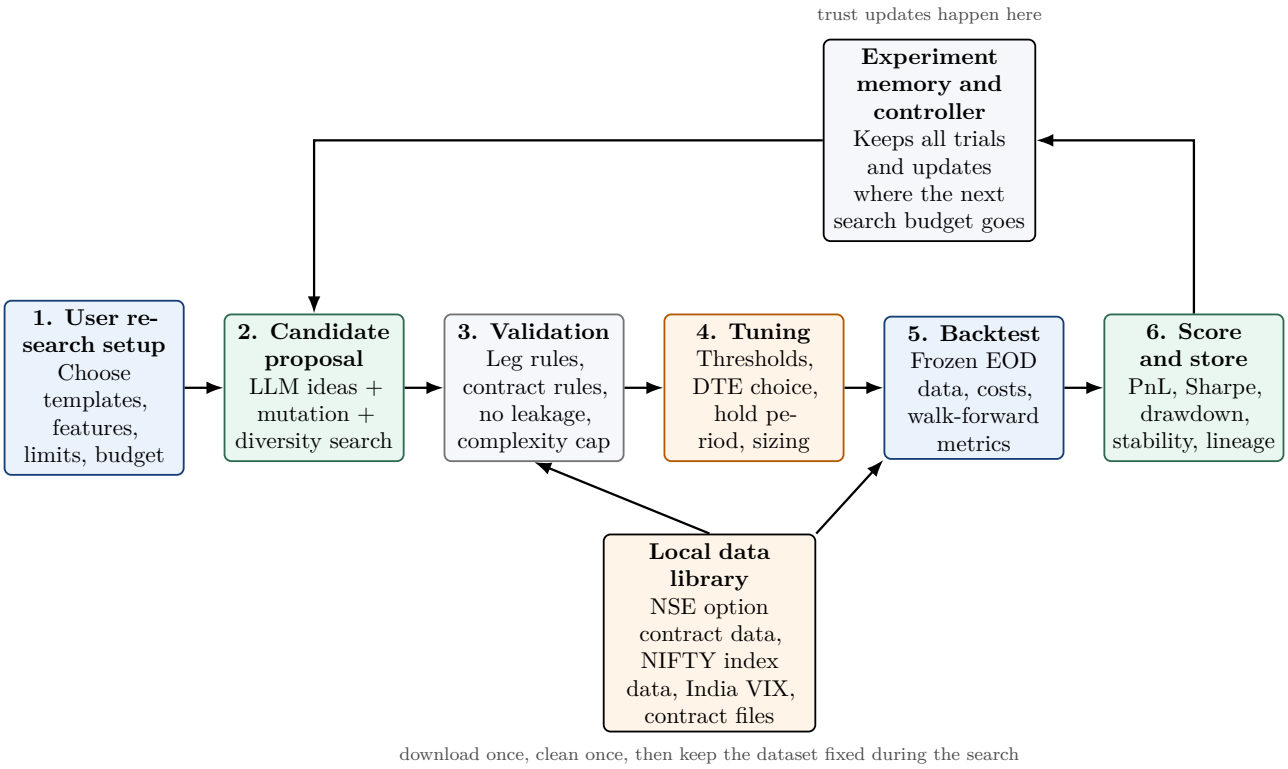
Trust-Calibrated Strategy Search for NIFTY Weekly Options

Project Proposal

Hardik Kalia Manasa Kalaimalai Aanvik Bhatnagar Keval Jain

Our project studies short-dated NIFTY options ideas on a fixed local dataset. The market, time period, data frequency, and basic contract universe are fixed by the project. The user only chooses among allowed trade templates, approved features, risk limits, and search settings. The system then creates valid candidate rules, tunes the numeric parts, backtests them on the frozen dataset, stores every result, and decides what to explore next. The main research question is about **trust**: how much search budget should be given to LLM-generated ideas, local improvements of past winners, and diversity search.

For ease of explanation, we first describe the project through a workflow diagram.



Example user input inside the fixed project setup

```
allowed_structures: [ATM straddle, 1-step call spread, 1-step put spread, 1-step risk reversal]
features: [atm_iv, iv_minus_rv20, skew_proxy, term_slope, nifty_ret1, nifty_ret5, rv20, india_vix_change, oi_change]
constraints: [max_legs=2, hold=1-3 trading days, liquidity_filter=on]
objective_mode: robust_sharpe
search_preference: balanced
budget: 400 candidate evaluations
```

Project-fixed NIFTY setup → user chooses valid search inputs → valid candidate rules → parameter tuning → EOD backtest → score + store → next round → top few strategies + explanation

1 Fixed project scope

We fix the project to one market and one data setup so that the scope stays manageable and the results remain meaningful.

Project choice	Decision for this proposal
Market	NSE NIFTY 50 weekly index options only
Data frequency	End-of-day only
Data window	1 Jan 2021 to 31 Dec 2025, frozen locally after download and cleaning
Main contracts	Nearest weekly expiry with 3–10 days to expiry, ATM to 1% OTM
Trade templates	ATM straddle, 1-step call spread, 1-step put spread, 1-step risk reversal
Maximum size of a candidate	At most 2 legs
Holding period	1 to 3 trading days
Execution rule	Build signal from day t data and enter on day $t+1$ using the EOD price rule used in the backtest
Data used by search loop	Only the frozen local dataset. No live download inside the loop
Cost model	Premium-based transaction cost plus a simple liquidity penalty
Final output	A shortlist of strong candidate rules, their settings, and a clear experiment log

Why this scope makes sense:

- NSE provides official historical pages for contract-wise derivative data, historical index data, and India VIX. This makes the data plan realistic.
- End-of-day data is a more practical choice. NSE also has separate exchange data products for order and trade level history.

2 What the user can and cannot input

The system is human-guided, but the user works within a fixed project scope. The user gives a **research setup** inside the fixed project scope. The market, dataset, date range, and EOD backtest rule are already fixed by the project.

Inputs we will allow

- Choose from the listed trade templates.
- Choose a subset of the approved features.
- Choose the objective mode, for example robust Sharpe or return with drawdown control.
- Set basic limits such as maximum legs, DTE band, hold period, and experiment budget.
- Ask the system to lean a little more toward exploration or refinement at the start.

Inputs outside the project scope

- Changing the market, date range, or data frequency.
- Minute-level or tick-level entries and exits.
- More than 2 legs.
- Any strike, any expiry, any instrument.
- External news, social media, or order book data.
- Raw Python code in the prompt.
- Tuning directly on the final holdout period.

Two concrete examples

Good example: *"Compare ATM straddles and 1-step call spreads using IV minus realized volatility, India VIX change, and short-term NIFTY momentum. Keep 3–10 DTE and 1–3 day holds."*
Bad example: *"Switch to BANKNIFTY, use minute data, search all option strategies, and maximize profit using any data you can find."*

3 Data plan and feature construction

3.1 Data we will store locally

The raw data plan is:

- **NSE contract-wise price-volume data** for NIFTY index options. This gives the contract-level option history we need for strikes, expiries, and prices.
- **NIFTY historical index data**. This is needed for moneyness, realized volatility, and Black-Scholes inputs.
- **India VIX history**. This gives an external market volatility regime feature.
- **Daily F&O contract files / common bhavcopy**, if needed, for contract metadata such as lot size and contract mapping.

After download and cleaning, all of this will be kept in a local versioned dataset. The search loop will only read from this frozen dataset.

3.2 Features we plan to start with

We plan to use the following families.

Feature family	Examples used in the project
Underlying move	NIFTY 1-day return, NIFTY 5-day return
Realized risk	5-day realized volatility, 20-day realized volatility
Option surface	ATM implied volatility, IV minus RV20, near-strike skew proxy, near-vs-next expiry term slope
Flow / liquidity	Open interest change, contract volume, simple liquidity filters
Regime	India VIX level and India VIX daily change

An important part of the project is that some of these features will be computed by us. In particular, implied volatility and simple Greeks can be computed from option prices with a Black-Scholes style formula and a simple interest-rate proxy. The exact rate choice can be finalized later.

3.3 How contracts will be selected

Each trading day, the backtest will:

1. find the nearest weekly NIFTY expiry with 3–10 DTE,
2. locate the ATM strike and nearby 1% OTM strikes,
3. build the allowed 2-leg templates from those contracts,
4. apply liquidity filters before a candidate can be tested.

The contract file carries the actual expiry date, so the system will use that metadata directly instead of hard-coding one weekday rule.

4 End-to-end workflow in more detail

4.1 Step 1: proposal stage

We will use three proposal sources.

- **LLM proposer:** suggests readable candidate rules from the allowed templates and features.
- **Mutation proposer:** takes a good past rule and changes one or two parts locally.
- **Diversity proposer:** adds a few valid but different candidates so the search does not become too narrow.

The LLM will not write free-form strategy code. It will only produce a structured rule inside our allowed grammar.

4.2 Step 2: validation stage

A candidate must pass a validation step before it reaches the backtester. Some examples:

- maximum 2 legs,
- only approved features and operations,
- no future data leakage,
- only contracts that actually exist in the local dataset,
- complexity cap, for example a limit on nested conditions or number of terms.

4.3 Step 3: parameter tuning

Once the structure is fixed, the system can tune the numeric parts. Typical knobs are:

- threshold levels,
- exact DTE inside the 3–10 day band,
- hold period inside the 1–3 day band,
- simple size or premium-budget choices,
- regime cutoffs for IV, VIX, or momentum filters.

4.4 Step 4: backtesting

The backtester is the main evaluation engine in the project. For each valid candidate, it will:

- build the signal using only information available up to day t,
- map the signal into one of the allowed trade templates,
- enter and exit with the EOD rule fixed for the project,
- mark the position through time,
- apply transaction costs and liquidity penalties,

- return PnL and diagnostics.

4.5 Step 5: scoring, storage, and the next round

Every run is stored, including:

- the candidate structure,
- its parameter values,
- backtest metrics,
- which proposal source created it,
- and how it was related to past candidates.

This stored history of past experiments helps the controller decide what to try next.

4.6 A simple two-round picture

Round 1. The user starts with the fixed setup. The LLM may suggest a long straddle rule activated when IV minus RV20 is very low. The mutation proposer may slightly change the hold period or use India VIX change as an extra filter. The diversity proposer may test a put spread in a very different regime. After backtesting, the system learns which family looks stronger.

Round 2. If the straddle family looked promising but only in a low-VIX regime, the controller can spend more of the next round on nearby straddle variants, while still keeping some budget for different spread ideas. This is where the trust part enters the project.

5 Core math and optimization questions

This is the most open part of the proposal. In the proposal we are not trying to fix every solution, but give a solid plan instead.

5.1 1. Search over structure plus parameters

A candidate can be written as a pair (s, θ), where s is the rule structure and θ is the set of numeric choices. At a high level, we want something like

$$(s^*, \theta^*) = \arg \max_{s \in \mathcal{S},\, \theta \in \Theta(s)} R(s, \theta).$$

Here R is a robust performance score. One possible form is

$$R = \text{Sharpe}_{\text{OOS}} - \lambda_{dd}\,\text{Drawdown} - \lambda_{to}\,\text{Turnover} - \lambda_c\,\text{Complexity}.$$

This exact score is not final. One option is a weighted score like the one above. Another is to treat return, drawdown, turnover, and complexity separately instead of combining everything into a single number.

5.2 2. Trust between proposal sources

The trust problem can be framed as budget allocation. If the next round has budget B_r, we choose weights for the three proposal sources:

$$B_r^{\text{LLM}}, \quad B_r^{\text{mut}}, \quad B_r^{\text{div}}, \qquad B_r^{\text{LLM}} + B_r^{\text{mut}} + B_r^{\text{div}} = B_r.$$

Possible approaches include:

- a bandit-style update based on robust hit rate,

- Bayesian reliability scores,
- a trust rule with a minimum exploration floor so no source gets zero budget,
- similarity-aware updates so that many near-duplicate rules do not count as many independent successes.

We want this part to be mathematically meaningful. The controller should learn, but it should also avoid getting stuck too early.

5.3 3. Parameter tuning and risk sizing

For parameter tuning, we are likely to compare a few practical methods such as grid search, random search, or Bayesian optimization tools. For sizing, one simple start is fixed-lot or fixed-premium sizing. A more advanced route is a small optimization problem with limits on delta, premium budget, or tail risk. The final choice here will depend on early experiments and implementation feasibility.

5.4 4. Statistical checks

Even good-looking backtests can be misleading. So the project should include:

- walk-forward or rolling validation,
- a final untouched holdout slice,
- sensitivity checks for transaction cost assumptions,
- stability checks across different volatility regimes,
- and at least a small correction for repeated testing.

6 Backtest and evaluation plan

A reasonable first split is:

- **Research and tuning period:** 2021–2024
- **Final holdout period:** 2025

Key metrics we want to record for every candidate:

- total return and average return,
- Sharpe ratio,
- maximum drawdown,
- hit rate,
- turnover,
- stability across sub-periods,
- sensitivity to cost assumptions,
- and rule complexity.

We do not plan a large comparison study between multiple systems. The main goal is to build one strong system and evaluate the quality and robustness of the strategies it finds.